

Agenda Médica

App Android nativo (Kotlin + Jetpack Compose) para agendamento de consultas médicas, com backend em Supabase (Postgres + Auth + Edge Functions + Realtime). Projeto de portfólio, construído do zero com o método [BMad](#): planejamento (PRD, arquitetura, UX) seguido de implementação por histórias, cada uma com spec própria, revisão em múltiplas camadas e verificação real contra o backend hospedado.

Visão geral

Paciente e Médico usam o mesmo app com papéis distintos:

- **Médico** se autocadastra (sem validação de CRM), define especialidade, convênios aceitos e o horário em que atende — dentro da janela fixa da clínica (08h-18h).
- **Paciente** se autocadastra, busca médicos por especialidade e localização (GPS ou manual) e agenda uma consulta de 30 minutos, com garantia de que dois pacientes nunca ocupam o mesmo horário mesmo sob concorrência.
- Consultas podem ser canceladas ou reagendadas por qualquer uma das partes até 24h antes; abaixo disso, bloqueio total, sem exceção.
- O Paciente recebe um lembrete por e-mail 24h antes da consulta; eventos de agendamento/cancelamento ficam registrados para uma futura notificação push.

Stack técnica

- **App:** Kotlin, Jetpack Compose, MVVM (View → ViewModel → Repository), [supabase-kt](#) (Auth, Postgres, Realtime, Functions). [minSdk](#) 26, [compileSdk/targetSdk](#) 37, AGP 9.2 (Kotlin embutido, sem plugin separado), Gradle 9.4.
- **Backend:** um único projeto Supabase (Postgres + Auth + Edge Functions + Realtime + [pg_cron/pg_net](#)), sem separação dev/produção.
- **Regras de negócio no banco, não só no app:** toda escrita de consulta passa por funções Postgres [SECURITY DEFINER](#); a garantia final contra conflito de horário é um índice único parcial, não uma checagem em memória.

- **E-mail transacional:** [Resend](#) (camada gratuita).
- **Testes:** JUnit para o app (lógica de domínio e ViewModels) e testes de UI Compose com Robolectric ([compose-ui-test](#), no mesmo [testDebugUnitTest](#)); scripts Node ([supabase/tests/*.mjs](#)) que provam as regras de concorrência e as funções do banco contra o projeto Supabase hospedado de verdade, sem mocks.

Estrutura do repositório

```
app/src/main/java/com/agendamedica/app/  
  data/                # Repository (único ponto de acesso ao Supabase) e provedores  
(localização, cliente remoto)  
  domain/              # Regras de negócio puras e testáveis (agenda, busca, modelos)  
  ui/                  # Compose: auth, doctor, patient, navegação, tema, componentes,  
splash  
  
supabase/  
  migrations/          # Schema, RLS e funções Postgres, uma migração por história  
(nunca editadas depois de aplicadas)  
  functions/           # Edge Functions (Deno): register-doctor, register-patient,  
send-reminders  
  tests/               # Scripts Node que provam as regras do banco contra o projeto  
hospedado  
  seed/                # Massa de demonstração (médicos, pacientes e consultas  
fictícios) para a base hospedada  
  
_bmad-output/  
  planning-artifacts/  # PRD, arquitetura (ARCHITECTURE-SPINE.md), UX  
(DESIGN.md/EXPERIENCE.md), epics.md  
  implementation-artifacts/ # Uma spec por história, com decisões de design e o  
histórico de revisão  
  test-artifacts/      # Checklist de teste manual (o que só um aparelho real  
pode confirmar)
```

Todo o histórico de planejamento e decisão de cada história está em [_bmad-output/](#) — é a fonte de verdade sobre *por que* o código é como é, não só *o que* ele faz.

Pré-requisitos

- Android Studio (ou apenas o JDK do seu SDK — este projeto foi desenvolvido usando o JBR do Android Studio como [JAVA_HOME](#), sem abrir a IDE) e um dispositivo/emulador Android.

- Uma conta no [Supabase](#) e um projeto criado (o desenvolvimento usa o projeto **hospedado** diretamente — não é necessário Docker nem `supabase start` local).
- [Node.js](#) (para a CLI do Supabase via `npx` e os scripts de teste em `supabase/tests/`).
- Uma conta no [Resend](#) para o lembrete por e-mail (opcional para rodar o app; necessário só para a Story 3.1 funcionar de ponta a ponta).

Configuração

1. Vincule o projeto Supabase:

```
npx supabase login  
npx supabase link --project-ref <seu-project-ref>
```

2. Aplique as migrações e publique as Edge Functions:

```
npx supabase db push  
npx supabase functions deploy register-doctor --use-api  
npx supabase functions deploy register-patient --use-api  
npx supabase functions deploy send-reminders --use-api
```

3. Configure os segredos das Edge Functions (nunca versionados):

```
npx supabase secrets set RESEND_API_KEY=<sua chave do Resend>  
npx supabase secrets set REMINDER_CRON_SECRET=<uma string aleatória>
```

O mesmo `REMINDER_CRON_SECRET` também precisa existir no Vault do Supabase como `reminder_cron_secret` (lido pelo job `pg_cron` que aciona o lembrete) — veja `supabase/migrations/0008_reminders.sql`.

4. Configure o app: copie `local.properties.sample` para `local.properties` (já ignorado pelo Git) e preencha:

```
SUPABASE_URL=https://<seu-project-ref>.supabase.co  
SUPABASE_ANON_KEY=<a chave "anon" do seu projeto>  
REMINDER_CRON_SECRET=<a mesma string do passo 3> # só usado pelos scripts de teste
```

5. Em Authentication → URL Configuration no painel do Supabase, adicione `agendamedica://reset-password` às Redirect URLs (necessário para a recuperação de senha).

Rodando o app

```
./gradlew assembleDebug
```

Instale o APK gerado (`app/build/outputs/apk/debug/app-debug.apk`) no celular, ou rode direto pelo Android Studio com um dispositivo/emulador conectado.

Testes

Testes de unidade (Kotlin: lógica de domínio, ViewModels e testes de UI Compose via Robolectric — rodam na JVM, sem emulador e sem tocar no backend):

```
./gradlew testDebugUnitTest
```

Scripts contra o projeto Supabase hospedado (provam concorrência, RLS e as regras de negócio no banco real; cada um cria e depois remove seus próprios dados de teste):

```
npm run test:concurrency          # exatamente um vencedor por horário disputado
npm run test:cancel-reschedule    # janelas de 24h/48h, papéis, reagendamento
npm run test:reminders            # claim atômico do lembrete, isolamento de falha
npm run test:notification-events  # RLS: nenhum cliente lê a tabela de eventos
```

Checklist de teste manual (o que só um aparelho real confirma — splash, GPS, e-mail de verdade, dois usuários simultâneos): `_bmad-output/test-artifacts/checklist-teste-manual.md`.

Decisões de arquitetura

As decisões centrais (e o porquê de cada uma) estão documentadas em [_bmad-output/planning-artifacts/architecture/.../ARCHITECTURE-SPINE.md](#).

Resumo das mais importantes:

- **Escrita sempre via função Postgres, nunca direto do cliente** — a constraint de unicidade no banco é a garantia final contra conflito de horário, não uma checagem em memória.
- **Disponibilidade é sempre computada, nunca armazenada como linha** — a grade de horários deriva da agenda do médico menos os horários ocupados, calculada em memória.
- **Lembrete disparado pelo próprio Supabase** ([pg_cron](#) + [pg_net](#)), nunca pelo cliente — reivindicação atômica evita envio duplicado sob execuções concorrentes.
- **Papel da conta verificado ao vivo pelo RLS**, nunca por um claim em cache no token.
- **Vocabulário de erro único** ([CONFLICT:](#) / [INVALID:](#) / [FORBIDDEN:](#)) em toda função do banco, mapeado para mensagens genéricas no app.

Estado do projeto

Todas as histórias dos Épicos 1-5 estão implementadas, revisadas e commitadas — cadastro/autenticação, busca e agendamento com garantia de concorrência, lembrete por e-mail e registro de eventos, identidade visual (splash/ícone) e a grade de horários corrigida (consulta de 30min + intervalo de 15min). Push notification é uma decisão deliberadamente adiada (o evento já fica registrado, pronto para um consumidor futuro). Publicação em loja não é escopo deste projeto — instalação é direta via APK.